Intelli-Gate Architecture

Status and scope

This repository implements an authenticated Node.js/Express web application with a browser-based dashboard simulation. It is not currently an IoT system: no Raspberry Pi peripheral driver, ESP32 firmware, hardware transport, sensor acquisition, physical alarm, or actuator interface is implemented. The dashboard readiness animation only checks the authenticated application API and prepares simulation controls.

The hardware documents describe a **proposed** edge system, not a built, electrically reviewed, or performance-validated product. No regulatory, electrical, privacy, screening-accuracy, or safety compliance claim is made here.

Implemented runtime

```text
Operator browser
  ├── Sign-in page and dashboard UI
  ├── CSRF-protected authentication and logout requests
  └── Authenticated readiness request
              │ same-origin HTTP; HTTPS required for network deployment
              ▼
Node.js / Express process (server.js)
  ├── Protected dashboard and API routes
  ├── Server-side session and role checks
  ├── Scrypt password verification and login rate limiting
  ├── data/auth-users.json
  ├── data/session-secret
  └── data/sessions.sqlite

Browser app.js
  └── Local demonstration state, simulated events, simulated alerts and controls

Hardware
  └── Not implemented; readiness API reports all listed devices not connected
```

`GET /api/system/readiness` returns application/session readiness with `mode: "simulation"` and `hardware.integration: "not-implemented"`. It does not probe the Pi, ESP32, GPIO, I²C bus, UART, sensors, coils, alarm, or actuator. “Ready” in this response means ready to run the software simulation only.

The browser is untrusted. Authentication and role authorization are enforced by Express. Browser-only role checks, hidden controls, simulation labels, or local event state must never be used as authorization for future readings, exports, configuration changes, or device commands.

Proposed hardware target — not implemented

The project materials describe a Raspberry Pi edge host, an ESP32 metal-sensing controller, temperature and passage sensors, local display/alarm, persistent event records, and an operator dashboard on a private LAN. If implemented, the existing Node/Express application should remain the authoritative web/authentication boundary unless a separately approved architecture decision changes it. Do not build a parallel Flask application simply because older diagrams depict Flask/Socket.IO; the checked-in runtime is Node/Express.

An eventual software boundary could be:

```text
Sensors / coil front-end
  ├── Raspberry Pi interfaces: temperature, passage beams, local LCD/alarm
  └── ESP32: locally scheduled coil sampling and channel health
                  │
                  │ versioned, bounded, validated device messages
                  ▼
Raspberry Pi edge services
  ├── Hardware adapter (not implemented)
  ├── Message validation, freshness, sequence/deduplication and fault state
  ├── Node/Express authenticated APIs and server-side authorization
  ├── Durable event store and audit trail (future hardware data model)
  └── Local dashboard delivery over a restricted network
                  │ HTTPS/WSS or an approved local transport
                  ▼
Authorized operator browser
```

This is a boundary proposal, not a pin-level design. The final selection between a Node-native Pi driver and a separately supervised local adapter process depends on the confirmed board, driver support, and deployment environment. In either case, the browser must not communicate directly with GPIO or send unauthenticated commands to an ESP32.

Device communication contract

The six-zone data dictionary proposes ESP32-to-Pi frames containing a protocol version, device code, boot identifier, monotonic sequence, event type, zone,

signal and unit, threshold, decision, timestamp, quality flag, and optional CRC. It explicitly says that the circuit does not establish a complete receive front-end or production protocol. Before implementation, define and test framing, maximum message size, encoding, version negotiation, checksum/integrity behavior, replay/deduplication, timeout, clock handling, reconnect, and malformed-message behavior. Reject and audit invalid, stale, duplicated, out-of-range, or unknown-version messages; never convert missing device data into a successful/green status.

UART is a physical data link, not cryptographic authentication. Protect the device enclosure and wiring; if the eventual transport is network-based, authenticate each device and protect the channel with an approved secure protocol and per-device credentials.

Zone and source-document reconciliation

The project owner has selected six physical positions: `L_UPPER`, `L_MIDDLE`, `L_LOWER`, `R_UPPER`, `R_MIDDLE`, and `R_LOWER`, as specified in the proposed [Six_Zone_Data_Dictionary.docx](./Data%20Dictionary/Six_Zone_Data_Dictionary.docx). That design excludes a bag-specific pathway. The revised Chapter 4 and older DFD/architecture material still describes four logical classes (`UPPER`, `MIDDLE`, `LOWER`, `BAG`) or refers to the bag zone. The checked-in circuit image depicts six proposed coil channels. Update those stale artifacts to the owner-selected six-zone contract; the six physical coils still require verification as six independent, accurate detection zones.

For software discussion, the six-zone data dictionary is the selected **logical data contract**, but not a final electrical/build specification. Before construction freeze, update the objectives, scope, mechanical drawing, BOM, diagrams, firmware mapping, dashboard labels, test protocol, and stored zone master data consistently. Six physical coils do not by themselves prove six independent, accurate detection zones.

The circuit image proposes a Raspberry Pi 4, ESP32 DevKit, MLX90614, two break-beam sensors, an I²C LCD, buzzer driver, six coil-driver channels, a 5 V/3 A rail, 24 V auxiliary rail, and servo/actuator. The project owner has confirmed a physical actuator is in scope; its exact model and safe-operation design remain unconfirmed. The proposed pin maps are examples only. Exact models, tolerances, drive/receive circuit, electrical ratings, channel independence, and production pin assignments remain unverified.

Readiness and safety behavior required for a future live mode

Simulation mode and hardware mode must remain unmistakably distinct. A future live readiness check should be based on real, bounded checks and recent

telemetry, with explicit `connected`, `degraded`, `fault`, and `not-connected` states per required device. It must not infer hardware health from server startup, session authentication, or the existence of a serial port. Startup must remain locked or visibly degraded if a required device fails its defined check.

At minimum, future event handling should:

- Preserve device identity, boot/session identity, monotonic sequence, event time, Pi receive time, quality, and fault state.
- Treat stale, ambiguous, noisy, saturated, malformed, or missing sensor data as unavailable/faulted rather than a pass.
- Keep local detection and alarm behavior safe if the browser, server, LAN, or device link fails.
- Require authenticated, server-authorized, auditable state changes and commands; separate operator permissions from device credentials.
- Avoid retaining unnecessary identity or health data. Temperature values are non-diagnostic screening observations and require approved measurement conditions, retention, and privacy policy.
- Never automatically actuate a barrier in a way that can block egress. Any physical actuator requires an independently reviewed risk assessment, safe state, manual release, and emergency override.

Hardware and compliance blockers

No physical connection or operation should begin until qualified reviewers resolve and document:

1. **Design authority:** update every inconsistent manuscript and diagram to the owner-selected six left/right zones; remove or explicitly rescope references to four logical classes and BAG.
2. **Bill of materials:** record exact Raspberry Pi, ESP32 board/revision, sensor/break-beam/LCD modules, coil assemblies, drivers, power supplies, protection components, enclosure, and actuator part numbers and datasheets.
3. **Electrical engineering:** review voltage domains, Pi 3.3 V GPIO limits, I²C pull-ups, ESP32 boot-strapping pins (the diagram includes GPIO12), interface isolation/level shifting, driver/transient ratings, coil current and heat, supply budget, grounding, fusing, enclosure, and mains safety. The proposed common-ground drawing and rail ratings are not validated.
4. **Sensing design:** select a complete metal detector topology including receive/sense acquisition, analog conditioning, sampling schedule, calibration, interference management, and detection thresholds. Coil excitation alone is not a validated metal detector.
5. **Functional and safety acceptance:** specify direction-counting ambiguity, temperature screening constraints, alarm behavior, fault handling, actuator safe

state/manual release, and measurable pass/fail criteria. The actuator is in scope but must not be operated before this review.
6. **Security, privacy, and site approval:** choose the deployment trust boundary, device identity and key provisioning, update/recovery process, LAN segmentation, event retention, access policy, incident response, and applicable institutional/jurisdictional requirements. Applicable requirements have not yet been identified.

Until these gates pass and testing evidence exists, this repository is a software simulation and architecture proposal. For setup and sign-off steps, see [INSTRUCTION.md](./INSTRUCTION.md) and [SECURITY.md](./SECURITY.md).